

Object Counting Across Modalities

Prepared for: Joana Owusu • **Date:** May 18, 2026 • **Focus:** Image, Video, Open-Vocabulary, and Emerging 3D Counting Paradigms

Building an in-depth survey paper that maps out over 1,000 highly related papers requires an optimized, automated pipeline. Relying on raw browser scraping of Google Scholar leads to severe IP blocks and unmanageable, unstructured text data. This document outlines a concrete, execution-ready, end-to-end framework to programmatically pull, organize, filter, and synthesize the object-counting literature from seed discovery to the final LaTeX compilation.

Phase 1: Seed Paper Discovery & Target Selection (Days 1–4)

The foundation of a massive-scale literature review relies on high-quality **seed papers**. Instead of performing a broad keyword query, execute 5 to 10 highly specific queries to find core architectural and modality benchmarks. Inspect only the top 20 relevance-sorted and top 10 date-sorted results per query to avoid information overload.

Core Bucket	Targeted Search Queries	Target Count	Key Benchmarks / Models
1. Survey & Taxonomy	"class-agnostic counting" survey, "visual counting" survey, "few-shot object counting" taxonomy	5–10 Papers	Recent CVPR/ ECCV/IEEE templates
2. Image Domain	"few-shot object counting", "exemplar-based counting", "open-vocabulary object counting", "object counting" SAM	20–30 Papers	FSC-147, FSCD-147, CountBench, OmniCount
3. Video Domain	"video object counting", "open-world object counting in videos", "unique instance counting" video	10–15 Papers	TAO-Count, VideoCount, MOT20-Count, CountVid
4. Extended Modalities	"3D object counting", "object counting" "point cloud", "RGB-D object counting", "remote sensing" counting, "cell counting" vision	10–15 Papers	ShanghaiTech, CARPK, Science-Count, 3D PointNet++ templates

Operational Milestone: seed_papers.csv

By the end of Day 4, manually save these 55–85 foundational papers into a flat `seed_papers.csv` file. This file must contain at minimum the following columns: title, authors, year, DOI, venue, and modality.

Phase 2: Programmatic Expansion & API Aggregation (Day 5)

With your seed list established, leverage **Claude Code** or **Codex** to bypass Google Scholar's scraper defenses. You will build a Python automation pipeline that calls official, rate-limit-friendly scholarly APIs to programmatically expand the 55–85 seeds into a network of over 1,000 highly relevant papers via forward and backward citation graphs.

The Scholarly Data Infrastructure

- **OpenAlex API** Free REST API containing global metadata. Excellent for downloading bulk publication documents without authorization tokens.
- **Semantic Scholar API** Excellent for uncovering AI-driven citation metrics and isolating papers marked explicitly as "highly influential citations."
- **Crossref & arXiv APIs** Critical for resolving missing DOIs, checking LaTeX source templates, and pulling the absolute latest preprints in computer vision.

The API Extraction Script Prompt

Feed the following absolute prompt into Claude Code or Codex to construct your data pipeline code:

```
import pandas as pd
# Instruction for Claude Code: Write a complete Python pipeline that:
# 1. Reads 'seed_papers.csv' containing columns: title, authors, year, DOI, venue,
modality.
# 2. For each paper, queries the Semantic Scholar and OpenAlex REST APIs.
# 3. Extracts: Abstract, Citation Count, Literature References, and Highly Influential
Citations.
# 4. Performs a forward lookup (all papers citing the seed) and backward lookup (all
references).
# 5. Automatically deduplicates results based on strict DOI matches and fuzzy title
strings (Levenshtein > 0.85).
# 6. Flags records missing crucial metadata into a separate 'missing_metadata_report.csv'.
# 7. Outputs a unified database called 'literature_matrix.xlsx' and a structured
'references.bib' file.
```

Phase 3: Automated Filtering & Literature Matrix Structure

The expansion phase will generate metadata for 1,500+ candidate papers. To distill this into the target core dataset, your script must automatically filter papers based on structured inclusion rules and map them to explicit matrix attributes.

Structural Inclusion / Exclusion Logic

- **Inclusion Rules:** The paper must actively present an architecture, loss function, dataset, or benchmark metric explicitly evaluated against visual counting metrics (MAE, RMSE, OBO, or Anchor-AP).
- **Exclusion Rules:** Eliminate generic object detection papers (e.g., standard YOLO/Faster R-CNN models) that only mention counting as a generic secondary downstream task but lack specialized density or class-agnostic matching evaluations.

Mandatory Literature Matrix Schema

Your exported `literature_matrix.xlsx` file must map each paper to the following exact tracking fields to ease cross-referencing during drafting:

Field Name	Valid Categories	Description / Purpose
Modality	image, video, 3D/point-cloud, remote-sensing, microscopy, thermal, event-camera	Isolates the data structure processed by the model's sensory encoders.
Task Type	density-based, detection-based, few-shot, zero-shot, open-vocabulary, tracking-based	Defines the algorithmic mechanism driving the count estimation.
Input Prompts	bounding boxes, point exemplars, natural language text, class names, unsupervised	Tracks the auxiliary conditioning variables required at inference time.
Output Format	scalar count, density map, bounding boxes, segmentation masks, trajectories	Determines the level of structural localization provided by the network.
Core Evaluation Metrics	MAE, RMSE, Grid-MAE, ID-Switches, Multi-Object Tracking Accuracy (MOTA)	Standardizes performance benchmarking across varying datasets.

Phase 4: Vector Clustering & Taxonomy Mapping

To organize the final 1,000+ papers without manual reading, use text embeddings to discover semantic themes across titles and abstracts. This exposes hidden research trajectories and structural sub-domains automatically.

```
# Semantic Clustering Script Template to generate the structure
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

# Extract all abstracts from your generated literature matrix
abstracts = dataframe['abstract'].fillna(dataframe['title']).tolist()
vectorizer = TfidfVectorizer(stop_words='english', max_features=2000)
X = vectorizer.fit_transform(abstracts)

# Perform K-Means clustering across your specialized taxonomy themes
num_clusters = 6
model = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
dataframe['cluster_id'] = model.fit_predict(X)
# Outputs thematic blocks: e.g., Open-Vocabulary Models, Video Tracking, Density Map Networks
```

Phase 5: High-Impact Survey Writing & Taxonomic Outline

An outstanding computer vision survey paper tracks structural paradigm shifts over time. Organize your outline to demonstrate the clear evolutionary leap from early closed-world CNN density estimation networks to modern multi-modal foundation models.

Recommended Structural Outline for the Paper

- 1. Abstract & Introduction:** The fundamental problem of counting under occlusion and scale variation. Limitations of classical detection vs. density estimation.
- 2. Taxonomy of Object Counting Paradigms:**
 - *Fully Supervised Counting:* Fixed-class CNNs, density regression networks.
 - *Class-Agnostic / Few-Shot Counting:* Exemplar matching, feature correlation maps (e.g., FSC-147 baselines).
 - *Zero-Shot & Open-Vocabulary Counting:* Vision-Language Models (VLMs), grounding transformers, adapting Segment Anything Models (SAM/SAM2/SAM3) via region-of-interest tiling.
- 3. Algorithmic Architectures Across Modalities:**
 - *Static Images:* Spatial feature pyramids, cross-attention networks, patch-matching.
 - *Video Sequences:* Tracking-by-counting, spatio-temporal attention, resolving temporal consistency, identity tracking across occlusion frames.

- *Emerging Multi-Modal Dimensions:* 3D point cloud projections, RGB-Depth matching, thermal and high-speed event-camera processing streams.

4. **Comprehensive Dataset & Benchmark Compendium:** Comparative tracking table detailing sample capacities, query prompts, and leading state-of-the-art architectures.
5. **Open Challenges & Future Outlook:** Edge-hardware deployment constraints (robotics, assistive vision devices), processing dense scenes without catastrophic memory overhead, and open-world generalization.

Execution Checklist for Publication Success

- **Verify BibTeX Integrity:** Ensure your Python API pipeline strips out unescaped special characters to prevent LaTeX compilation breaks.
- **Identify Active Trajectories:** Highlight the dramatic rise in publications adapting foundational vision systems (like SAM) for dense, interactive scene queries.
- **Maintain a Balanced Modality Matrix:** Ensure specialized applications (microscopy, remote sensing) are given clear sub-sections within the broader multi-modal narrative.